

Guía de Ejercicios 1

Tecnología Digital VI

Juan Ignacio Elosegui

Universidad Torcuato Di Tella
Marzo 2026

Introducción

Ejercicio 3

- **Descriptivos:** ¿qué está pasando? Se busca entender y resumir los datos existentes, sin hacer predicciones.
- **Predictivos:** ¿qué va a pasar? Se usan datos históricos para predecir datos futuros. Es el problema típico en machine learning.
- **Prescriptivos:** ¿qué debería hacer? Se recomiendan acciones usando predicciones más optimización de decisiones.

El problema de la predicción de abandono de un cliente es predictivo.

Ejercicio 4

- **Aprendizaje supervisado:** en el dataset se cuenta con features y labels. Se quiere encontrar algún tipo de función que explique cómo los features explican el label.
- **Aprendizaje no supervisado:** en el dataset se cuenta únicamente con features. Se quiere encontrar estructuras o patrones en estos datos.
- **Aprendizaje por refuerzos:** el dataset no es más fijo, sino que son problemas en los cuales se deben elegir decisiones para llegar a outputs deseables.

Ejercicio 5

En regresión se busca predecir una variable numérica, como puede ser un número; mientras que en un problema de clasificación se busca predecir una variable categórica. Las variables numéricas son continuas mientras que las variables categóricas son discretas.

Preprocesamiento de datos

Ejercicio 6

Inciso (a)

Si el dataset es muy pequeño como para imputar valores con la media, esta no será realista. Podría ser una buena idea si se tienen muchos datos como para tener una media bien

definida y robusta. Por otro lado, si el dataset es grande, no es mala idea quitar las filas con faltantes, ya que el dataset es robusto a cambios si es que no tiene muchas filas con faltantes.

Inciso (b)

Ejercicio 7

Inciso (a)

Si se usa un modelo basado en distancias, como KNN o K-means, el modelo estará dominado por la variable de mayor escala, **Sueldo Anual**. Esto pasará porque las distancias euclídeas dependen del valor absoluto, y **Sueldo Anual** tiene valores muchos más altos que **Edad**.

Inciso (b)

- **Min-Max Scaling**: se reescalan los datos en un rango fijo, casi siempre $[0, 1]$, para mantener la forma de la distribución. Esto es sensible a outliers, porque un valor extremo aplastará al resto.
- **Standard Scaling (Z-score)**: se centran los datos según la media y el desvío estándar. Este scaling se ve menos afectado por la escala real de los datos, y funciona mejor si los datos son casi normales.

La diferencia principal es según en qué criterio se basa para estandarizar.

Ejercicio 8

Inciso (a)

Podría ser perjudicial porque introduce una ordinalidad que no es correcta y además es arbitraria. Básicamente quiere decir que **Rojo** < **Verde** porque $0 < 1$.

Inciso (b)

One-Hot Encoding transforma cada categoría en una variable binaria independiente para que no haya relaciones ordinales.

Ejercicio 9

Inciso (a)

Los **filter methods** seleccionan variables usando métodos estadísticos independientes del modelo, mientras que los **wrapper methods** evalúan subconjuntos de variables entrenando y validando un modelo.

Inciso (b)

El problema es el costo computacional, en especial si un dataset tiene muchas variables.

Ejercicio 10

- **Oversampling**: se aumenta la cantidad de observaciones de la clase minoritaria, ya sea duplicándolos o generándolos. No se pierde información y mejora la representación de la minoría, pero puede generar overfitting y aumentar el tamaño del dataset.
- **Undersampling**: se reduce la cantidad de observaciones de la clase mayoritaria removiéndolas. En general es más rápido de entrenar porque es más pequeño el dataset y termina más balanceado, pero se pierde información valiosa.

Ejercicio 11

Inciso (a)

El error es que se hizo el scaling luego de dividir el dataset. Esto se debe hacer siempre después. Esto va a generar **data leakage**.

Inciso (b)

El **data leakage** sucede cuando se filtra información del test set al training set. El modelo ya vio cómo es el test, entonces las métricas van a ser muy optimistas. Primero se divide el dataset en train y test, y luego se hace el scaling.

Model Selection

Ejercicio 12

Overfitting es cuando el modelo aprende demasiado los detalles más finos del dataset. Funcionará muy bien en training pero lo hará muy mal en testing.

El **underfitting** se da cuando el modelo no puede aprender los patrones relevantes del dataset. Un modelo con underfitting funcionará mal en training y testing.

Ejercicio 13

En el **training set** se entrena el modelo, en **validation set** se “prueba” el modelo antes de utilizarlo para saber su rendimiento de antemano, y el **test set** es un conjunto de datos no visto sobre el cual se busca predecir usando el modelo que mejor rindió en el conjunto de validación.

Si usásemos únicamente un *holdout set* estaríamos usando el modelo a ciegas sin saber a priori si funciona bien o mal.

Para poner un ejemplo: nosotros armamos un auto y directamente se lo vendemos al cliente. Probablemente el cliente se mate si no probamos el auto antes. Análogamente, armar el auto es el training set: vemos que todas las partes estén en condiciones; probarlo dentro del taller es el validation set: nos fijamos que las ruedas giren, que el motor pueda hacer la combustión; y sacar el auto a la calle es el test set: queremos que funcione siempre bien.

Ejercicio 14

K-fold Cross-Validation consiste en dividir el dataset en k folds (particiones), para que se pueda entrenar en $k - 1$ y validar en el k donde no se entrenó. Esto tiene la ventaja de tener una evaluación más robusta (porque no depende de una sola partición), se reduce el azar (porque no depende de cómo se partitionaron los datos) y se consigue una mejor estimación de la performance en testing.

Ejercicio 15

Leave-one-out Cross-Validation tiene la *ventaja* de que se aprovechan al máximo las observaciones del dataset para entrenar el modelo en cada iteración, pero la *desventaja* aparece cuando tenemos un dataset con muchas observaciones, porque se dispara la complejidad computacional.

Ejercicio 16

Las particiones aleatorias estándar pueden introducir *data leakage* o distorsionar la estructura del problema, por lo que se pueden usar las siguientes estrategias:

1. **Stratified K-Fold.** Se usa cuando el dataset está *desbalanceado*, que es justamente lo que soluciona. Sería un problema si por fold haya muchas o pocas observaciones de la clase mayoritaria o minoritaria.
2. **Time Series Split.** Se usa cuando los datos tienen un *orden temporal*. Estas observaciones no se deben mezclar de manera arbitraria, porque se usaría información del futuro para predecir el pasado, derivando en data leakage.
3. **Leave One Group Out.** Se usa cuando los datos están agrupados bajo algún criterio. Evita que los datos del mismo grupo aparezcan en entrenamiento y en validación, cosa que resultaría en métricas infladas.

Métricas

Ejercicio 17

Lo que interesa es la métrica *recall*, porque nuestro modelo debe poder capturar la mayor parte de los positivos entre los positivos verdaderos detectados más los falsos negativos (que serían los que se “pasan”).

Precision se puede interpretar como las veces que nuestro clasificó a alguien como un paciente con alta probabilidad de desarrollar la enfermedad, y estos *realmente* eran propensos a enfermarse.

Recall es cuántos pacientes con alta probabilidad de desarrollar la enfermedad *reales* pudo capturar nuestro modelo.

Ejercicio 18

En datasets altamente desbalanceados, la curva ROC puede resultar engañosa porque la tasa de falsos positivos (FPR) se calcula sobre una gran cantidad de negativos, lo que

hace que incluso un número considerable de falsos positivos produzca un FPR bajo. Esto devolverá estadísticas muy optimistas de más.

En cambio, la curva precision-recall es más sensible a los falsos positivos, ya que la precision penaliza directamente su presencia. Por ello, en problemas donde la clase positiva es minoritaria y de mayor interés, la curva precision-recall resulta más informativa para evaluar el rendimiento del modelo.

Ejercicio 19

- **RMSE.** Prioriza los errores *grandes* (porque está elevado al cuadrado). Se puede usar en modelos de predicción de salario, donde el mismo es positivo y puede haber valores extremos a los cuales esta métrica es sensible.
- **RMSLE.** Prioriza los errores *en su proporción*. Es útil en problemas donde importa más el error porcentual que el error absoluto, como en predicción de crecimiento o cantidades que varían en órdenes de magnitud.

Ejercicio 20

- **MAE.** Se prioriza el error absoluto promedio. Esto puede servir para predicción de distancias.
- **MAPE.** Se prioriza el error relativo. Equivocarse en un valor pequeño es mucho más grave que equivocarse en un valor grande.

Ejercicio 21

Inciso (a)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Accuracy} = \frac{99,000 + 0}{99,000 + 0 + 1,000 + 0}$$

$$\text{Accuracy} = \frac{99,000}{99,000 + 1,000}$$

$$\text{Accuracy} = \frac{99,000}{100,000}$$

$$\text{Accuracy} = 0,99$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Precision} = \frac{99,000}{99,000 + 1,000}$$

$$\text{Precision} = 0,99$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{99,000}{99,000 + 0}$$

$$\text{Recall} = 1$$

Inciso (b)

Porque da una precisión del 99 % sobre un modelo que clasifica que una clase es positiva siempre.

Inciso (c)

El F1-Score utiliza la *media armónica* entre precision y recall porque esta penaliza fuertemente los casos en los que una de las dos métricas es muy baja. En cambio, la *media aritmética* puede dar un valor relativamente alto aunque una de las dos sea muy mala, dando una impresión engañosa del rendimiento del modelo.

La idea es que, para que el F1-Score sea alto, tanto la precision como el recall deben ser altos al mismo tiempo. Si una de las dos cae mucho, el F1-Score también cae de manera importante.

Por ejemplo, si un modelo tiene:

$$\text{Precision} = 1 \quad \text{y} \quad \text{Recall} = 0,01$$

la media aritmética sería:

$$\frac{1 + 0,01}{2} = 0,505$$

lo cual podría sugerir erróneamente un desempeño aceptable.

En cambio, el F1-Score sería:

$$F1 = 2 \cdot \frac{1 \cdot 0,01}{1 + 0,01} \approx 0,0198$$

Este valor refleja mucho mejor que el modelo en realidad tiene un rendimiento muy pobre, ya que aunque su precision sea perfecta, casi no logra recuperar positivos reales.

Ejercicio 22**Inciso (a)**

El enfoque que más prioriza el rendimiento sobre las clases minoritarias es el **Macro-averaging**, porque calcula la métrica por separado para cada clase y luego promedia todas por igual, tenga la cantidad de observaciones que tenga. Así, las clases minoritarias como **Pantalones** y **Camperas** tienen el mismo peso que la clase mayoritaria **Remeras**.

El **Micro-averaging** suma globalmente los verdaderos positivos, falsos positivos y falsos negativos de todas las clases antes de calcular la métrica, por lo que las clases mayoritarias dominan el resultado final. Por eso, el Micro-averaging refleja más el rendimiento global del modelo, mientras que el Macro-averaging permite evaluar mejor si el modelo también funciona bien en las clases minoritarias.

Inciso (b)

El **Micro-F1**, el **Micro-Precision**, el **Micro-Recall** y el **Accuracy** global toman exactamente el mismo valor en un problema de *clasificación multiclase de etiqueta única*, es decir, cuando cada instancia pertenece a una sola clase y el modelo predice exactamente una sola clase por ejemplo.

Esto pasa porque en ese caso cada error cuenta simultáneamente como un falso positivo para una clase y un falso negativo para otra, mientras que cada acierto suma un verdadero positivo. Al agregar globalmente todas las clases, las expresiones de Micro-Precision y Micro-Recall coinciden entre sí y también con el Accuracy, por lo que el Micro-F1 termina tomando ese mismo valor.